

DCFA: Dual-view Causal-inspired Attribution for Failure Reasoning in LLM-based Multi-agent Systems

Zehao Wang^{1,2}, Lanjun Wang^{2*}, Shilong Jin^{1,2}, Junjie Chen¹, Yanghua Xiao^{3,4}

¹College of Intelligence and Computing, Tianjin University, Tianjin, China

²School of New Media and Communication, Tianjin University, Tianjin, China

³College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China

⁴ Shanghai Key Laboratory of Data Science, Shanghai, China

{wzhslh, jinshilong_2025}@gmail.com

{wanglanjun, junjiechen}@tju.edu.cn, shawyh@fudan.edu.cn

Abstract

Large language model (LLM)-based multi-agent systems have experienced rapid growth in recent years. Despite their promise, such systems remain fragile, frequently exhibiting reasoning and coordination errors that can lead to system-level failures. Failure attribution in such systems relies on tracing natural language interactions among agents to identify the decisive error, which refers to the earliest action whose correction can reverse system failure. There are two key challenges: 1) Shallow attribution: Existing methods often capture only minor deviations, such as incomplete retrievals or formatting errors, which verification mechanisms can correct, while missing the decisive cause of system failure. 2) Contextual degradation: As the length of the system traces increases, the model’s reasoning ability rapidly deteriorates. To address these challenges, we propose DCFA, a training-free framework for failure attribution. DCFA integrates a global module that constructs structured causal-inspired dependency graphs from system traces to identify the initial decisive error, and a local module that applies local counterfactual-inspired reasoning to refine causal-inspired attribution. Experiments on the Who&When benchmark across six LLMs show that DCFA improves step-level accuracy by up to 8.27% over state-of-the-art baselines. Code is available in the public repository (<https://github.com/wzhSteve/DCFA>).

1 Introduction

With the emergence of large language models (LLMs), multi-agent systems (MAS) built upon LLM backends have rapidly evolved (Becattini et al., 2025; Ronanki, 2025). These systems enable autonomous collaboration among specialized agents for tasks such as code generation, knowledge search, and data analysis (Huang et al., 2024;

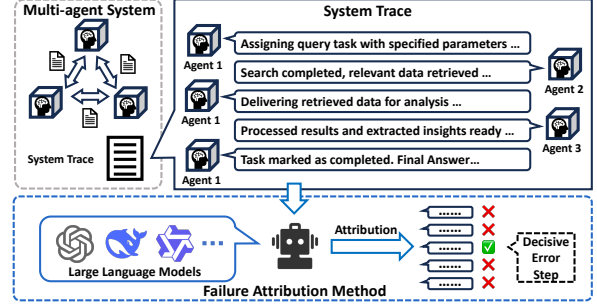


Figure 1: Illustration of failure attribution in an LLM-based multi-agent system. The system trace is represented as a textual record composed of interactive messages generated by agents.

Pan et al., 2024; Li et al., 2024; Majdoub et al., 2025). Despite the impressive progress in orchestration and tool integration, LLM-based multi-agent systems remain susceptible to coordination and reasoning failures (Cemri et al., 2026; Hammond et al., 2025; Shen et al., 2026). Errors such as misalignment in agents’ collaborations or improper tool usage can propagate across different agents along the system trace, leading to cascading reasoning errors that ultimately derail entire workflows (Cemri et al., 2026; He et al., 2025). This fragility highlights the urgent need for failure attribution in LLM-based multi-agent systems (Cemri et al., 2026; Hammond et al., 2025), which refers to the process of identifying the earliest actions whose correction could reverse system failure, known as decisive errors.

Recent efforts have begun to explore failure attribution in LLM-based multi-agent systems using LLMs as diagnostic detectors (Zhang et al., 2025, 2026). Existing approaches can be broadly classified into fine-tuning-based (Zhang et al., 2026) and instruction-based (Zhang et al., 2025; Cemri et al., 2026) paradigms. The fine-tuning-based approach, exemplified by AgenTracer (Zhang et al., 2026), constructs specialized datasets to train dedicated models for failure analysis. In contrast, instruction-

* Corresponding Author.

based methods (Zhang et al., 2025; Cemri et al., 2026) rely on carefully designed prompts to guide LLMs in analyzing multi-agent system traces and identifying the decisive errors. However, both paradigms face inherent limitations. Fine-tuning-based approaches offer strong controllability but require extensive annotation and repeated training, resulting in high costs. Instruction-based methods avoid these costs under a training-free constraint by leveraging LLMs’ general reasoning ability, yet they often fail to reliably interpret long, multi-step, and unstructured system traces, leading to inaccurate failure attribution.

In this work, we study failure attribution in LLM-based multi-agent systems under the training-free setting to improve the extraction of decisive errors from unstructured multi-step traces. We identify two fundamental challenges: 1) Shallow attribution: Existing LLM-based attribution methods often focus on minor, recoverable deviations (e.g., incomplete retrieval or formatting errors (Cemri et al., 2026; Banerjee et al., 2025)) that do not determine the final outcome. In contrast, the decisive error refers to the earliest action whose correction would reverse the system failure, truly driving the system to fail. For example, an agent may fabricate missing content when critical information is lost during the initial collection or retrieval process. Unlike missing information that can be subsequently verified and retrieved through downstream interactions, the fabrication error is difficult to verify and thus persists throughout the reasoning chain, misleading the system’s subsequent decisions. 2) Contextual degradation: As trace length and complexity increase, attribution performance degrades sharply. For instance, the Who&When benchmark (Zhang et al., 2025) reports that the step-level attribution accuracy of state-of-the-art methods decreases substantially with growing trace length with averaging around 15% for traces of length 10, but dropping to nearly 0% for those approaching length 100.

To overcome these challenges, we propose a training-free framework, named the Dual-view Causal-inspired Failure Attribution (DCFA). DCFA comprises two key modules: the Global Causal-inspired Attribution (GCA) module and the Local Counterfactual-inspired Enhancement (LCE) module, which jointly perform global reasoning over a causal-inspired dependency structure and local counterfactual-inspired validation. To move attribution beyond surface-level deviations, GCA constructs a structured causal-inspired dependency

graph and performs reasoning along it to uncover deeper dependency relationships underlying system failure. GCA extracts candidate minor deviations from the interaction system trace and builds a step-level causal-inspired dependency graph centered on these candidates. By performing global reasoning over the causal-inspired dependency graph across the entire system trace, GCA generates an initial hypothesis of the decisive error. To mitigate long-context degradation, LCE refines the global hypothesis through localized counterfactual-inspired reasoning. Starting from the hypothesized error, LCE performs a bidirectional search over the causal-inspired dependency graph to identify local dependency chains through which the error propagates, and evaluates how local corrections propagate through the reasoning chain and affect the final outcome via counterfactual-inspired evaluation. By evaluating these locally induced outcome changes, LCE progressively identifies the interaction that most plausibly constitutes the decisive error. In summary, our contributions are fourfold:

- We propose DCFA, a training-free framework for failure attribution in LLM-based multi-agent systems, integrating global reasoning with local counterfactual-inspired validation.
- To address shallow attribution, we construct causal-inspired dependency graphs to model interaction-level dependencies, enabling LLMs to move beyond minor deviations and identify decisive errors.
- To mitigate contextual degradation, we introduce localized counterfactual-inspired refinement that focuses reasoning on critical dependency chains, thereby improving attribution accuracy on long traces.
- Experiments on the Who&When benchmark with six LLMs show consistent gains, improving step-level accuracy by up to 8.27% over the state-of-the-art baselines.

2 Related Works

2.1 LLM-based Multi-Agent Systems and System Failure

Recent advances in LLM-based agents have driven rapid progress in multi-agent systems (MAS), enabling complex, long-horizon tasks through task decomposition and inter-agent communication (Yang et al., 2026; Sun et al., 2025). Prior work has proposed systematic taxonomies of agent architectures (Wu et al., 2023; Fourney et al., 2024),

identifying components such as role assignment, planning, external memory, and feedback mechanisms, which aim to bolster the problem-solving capabilities of LLM-based multi-agent systems.

Despite their promise, LLM-based multi-agent systems remain brittle (Zhang et al., 2025; Cemri et al., 2026). Empirical studies reveal frequent coordination failures and cascading reasoning errors, particularly in long or highly interactive execution traces (Liu et al., 2024b). These observations highlight the need for methods to analyze the decisive error of system failure.

2.2 Failure Attribution in LLM-based Multi-agent Systems

Failure attribution in multi-agent systems aims to identify the earliest mistake whose correction could reverse the overall system failure, referred to as the decisive error (Cemri et al., 2026; Fourney et al., 2024). Unlike prior work such as AgentErrorBench with AgentDebug (Zhu et al., 2025), which locates mistakes that trigger cascading reasoning distortions, or TRAIL (Deshpande et al., 2025), which identifies and analyzes all errors in trajectories, focusing on the decisive error directly targets the most actionable point for repairing task failure.

Recent approaches employ LLMs directly as diagnostic tools, including fine-tuning-based methods (Zhang et al., 2026) and instruction-based methods (Zhang et al., 2025; Cemri et al., 2026). Fine-tuned models offer strong control but require costly annotation and retraining. Instruction-based methods instead analyze execution traces via prompting. For instance, ECHO (Banerjee et al., 2025) is equipped with hierarchical context representations and multi-perspective consensus to improve attribution performance, while A2P (West et al., 2025) is designed to perform causal-guided attribution through prompt-based instruction.

Despite their effectiveness, existing methods rely heavily on LLM intrinsic reasoning, often resulting in shallow attribution and degraded performance on long traces. To address these limitations, we propose a unified framework that combines structured reasoning over a causal-inspired dependency graph with localized dependency-chain analysis to improve failure attribution.

2.3 Causal Reasoning and Counterfactual Analysis with LLMs

Recent work has explored LLMs for causal reasoning and counterfactual-inspired analysis, show-

ing that they can extract events, propose candidate causal relations, and simulate hypothetical interventions from unstructured text (Cheng et al., 2025; Luo et al., 2024). To improve robustness, several studies combine LLM-derived causal hypotheses with algorithmic validation or graph-based aggregation (Tong et al., 2024; Ban et al., 2025). Others treat LLMs as informative priors for causal-inspired dependency graph discovery, leveraging their encoded commonsense knowledge to guide structure learning (Darvariu et al., 2024; Jiralerpong et al.). Counterfactual simulation has also been used to probe the sensitivity and faithfulness of LLM reasoning chains (Tutek et al., 2025; Yu et al., 2025). These advances motivate our use of causal-inspired scaffolding and localized counterfactual-inspired validation for failure attribution in LLM-based multi-agent systems. Discussions of related work are deferred to Appendix J.

3 Problem Formulation

To respond to a user query, a multi-agent system generates a system trace including a sequence of interactions denoted as $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_n\}$, where each interaction $\tau_t = (a_t, c_t)$ consists of the agent identifier a_t and its corresponding content c_t . The MAS produces a final output $\hat{Y}$, which is expected to match the ground-truth outcome Y . A system failure occurs when the generated output deviates from the expected one, i.e., $\hat{Y} \neq Y$. Such failures may arise from one or multiple erroneous or misleading interactions within $\mathcal{T}$.

The objective of failure attribution is to identify the decisive error responsible for the system failure. When a single erroneous interaction accounts for the failure, the decisive error corresponds to the interaction whose correction can recover the correct outcome. When multiple errors jointly contribute to the failure and no single correction fully recovers the correct outcome, we define the decisive error as the interaction whose correction yields the largest reduction in the system failure.

Formally, a failure attribution model $\mathcal{M}$ is defined as:

$$\mathcal{M} : (\mathcal{T}, \hat{Y}) \mapsto \tau^*, \quad (1)$$

where τ^* denotes the ground-truth decisive error, i.e., the interaction whose correction either recovers the correct outcome when a single-step correction suffices, or maximally reduces system failure when multiple errors jointly contribute to the failure.

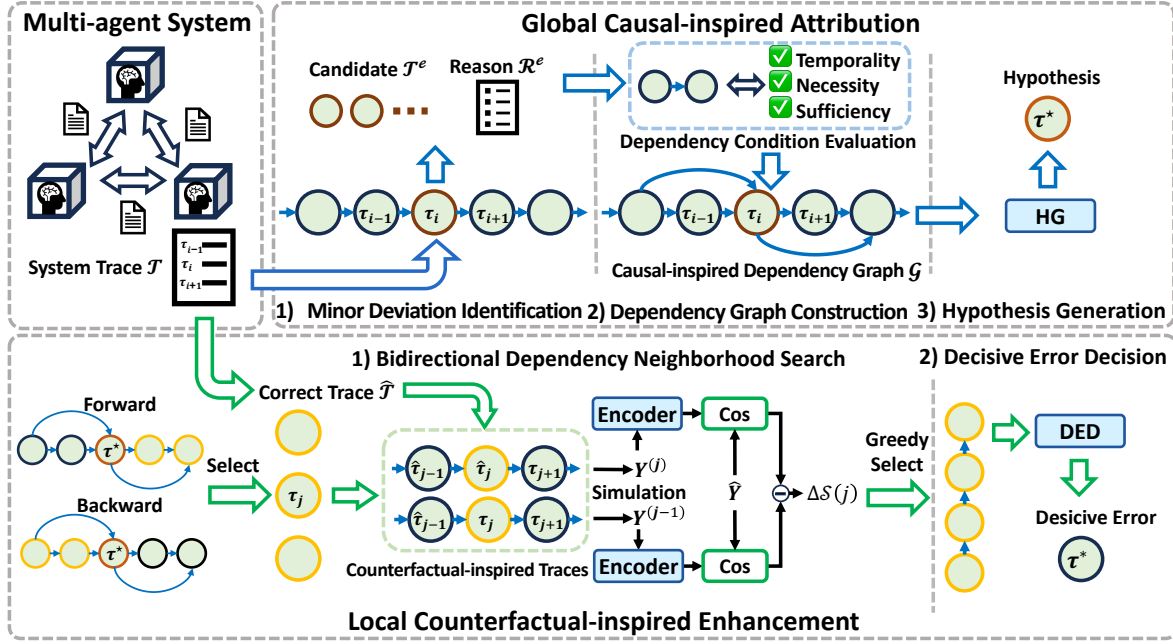


Figure 2: Overview of DCFA. DCFA integrates global causal-inspired attribution and local counterfactual-inspired enhancement to identify and refine the decisive error by reasoning over a causal-inspired dependency graph constructed from the system trace.

4 Methodology

In this section, we introduce the proposed DCFA framework (Fig. 2), which consists of the GCA (Sec. 4.1) and LCE (Sec. 4.2) modules. To overcome shallow attribution, GCA reasons over full system traces to uncover latent dependency relationships, constructing a causal-inspired dependency graph and identifying an initial hypothesis of the decisive error from a global perspective. Then, to further mitigate contextual degradation, the LCE module locally refines this hypothesis through counterfactual-inspired evaluation, using LLM-mediated approximate interventions to assess how correcting candidate interactions may alter the final outcome. This strategy first establishes a global causal-inspired dependency structure and then locally validates its critical links, enabling more interpretable failure attribution for unstructured and long system traces.

4.1 Global Causal-inspired Attribution Module

Interactions in system traces influence system outcomes through chained dependencies, requiring attribution methods that capture both minor deviations and how their effects propagate through the trace. Directly analyzing unstructured traces, LLMs often overemphasize salient deviations with-

out sufficiently considering their downstream dependencies and influence on the final outcome. To address this, GCA detects candidate deviations, constructs a directed causal-inspired dependency graph that links them to the full trace, and leverages graph-aware global reasoning to generate an initial hypothesis of the decisive error.

4.1.1 Minor Deviation Identification

Minor deviations are directly observable but typically benign faults in system traces, such as malformed outputs or incomplete information retrieval (Cemri et al., 2026; Hammond et al., 2025). These deviations are often recoverable through built-in verification, tool re-invocation (Barta et al., 2025; Zhou et al., 2025). However, minor deviations are not necessarily decisive errors, where they may serve as early symptoms of deeper failures.

Based on this motivation, GCA first extracts a set of candidate minor deviations that represent plausible manifestations of incorrect behaviors. Rather than prematurely determining which interaction constitutes the decisive error, GCA aims to retain all potentially relevant interactions for subsequent structured reasoning and refinement over the causal-inspired dependency graph.

Specifically, a designed process $MDI(\cdot)$ is utilized to detect the interactions of minor deviations and produce short natural-language explanations

for each identified minor deviation:

$$(\mathcal{T}^e, \mathcal{R}^e) = \text{MDI}(\mathcal{T}), \quad (2)$$

where $\mathcal{T}^e = \{\tau_1^e, \tau_2^e, \dots\}$ is the set of candidate interactions that are identified as minor deviations, $\mathcal{R}^e = \{r_1^e, r_2^e, \dots\}$ are the textual reasons associated with each candidate.

4.1.2 Dependency Graph Construction

Having identified the candidate minor deviations and their corresponding textual reasons, GCA constructs a structured causal-inspired dependency graph $\mathcal{G}$ that encodes plausible dependency relationships among interactions in the system trace $\mathcal{T}$. The construction is centered on the detected minor deviations $\mathcal{T}^e$ and their associated reasoning cues $\mathcal{R}^e$, ensuring that the resulting structure reflects interpretable, error-centric reasoning pathways.

Formally, the causal-inspired dependency graph is denoted as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of step indicators t corresponding to all interactions, and $\mathcal{E}$ is the set of directed edges representing putative dependency relationships. Specifically, an edge $e \in \mathcal{E}$ is denoted as (p, q) , indicating a directed dependency from interaction τ_p to τ_q , where τ_p precedes and potentially influences τ_q .

Dependency Condition Evaluation. A directed edge $e : \tau_p \rightarrow \tau_q$ is admitted into $\mathcal{E}$ only if it satisfies a set of structured dependency criteria (Zeng et al., 2026; Goldberg, 2019). For each ordered pair (τ_p, τ_q) with step indices $p < q$, GCA evaluates three complementary properties: temporality, necessity, and sufficiency.

Temporality enforces the directional ordering constraint that a preceding interaction must occur before the downstream interaction:

$$\text{Temp}(\tau_p \rightarrow \tau_q) = \mathbb{I}[p < q]. \quad (3)$$

Necessity tests whether the τ_q would fail to occur if the upstream step τ_p is removed, using a counterfactual-inspired judgment function J :

$$\text{Nec}(\tau_p \rightarrow \tau_q) = \mathbb{I}(J(\text{do}(\neg\tau_p)) = \text{No}). \quad (4)$$

Sufficiency assesses whether enforcing τ_p is sufficient to plausibly induce the outcome at τ_q :

$$\text{Suff}(\tau_p \rightarrow \tau_q) = \mathbb{I}(J(\text{do}(\tau_p)) = \text{Yes}). \quad (5)$$

An edge is added only when the temporal, necessity, and sufficiency conditions are all satisfied. This conservative rule filters out spurious correlations, retaining causally supported dependencies.

Causal-inspired Structured Reasoning. Given the structured dependency criteria, GCA performs causal-inspired structured reasoning over the system trace $\mathcal{T}$, focusing on the identified minor deviations $\tau^e \in \mathcal{T}^e$ to construct the causal-inspired dependency graph $\mathcal{G}$. The graph is constructed by incorporating these criteria into instruction sets $\mathcal{I}^c$:

$$\mathcal{G} = \text{CGC}(\mathcal{T}, \mathcal{T}^e, \mathcal{R}^e, \mathcal{I}^c), \quad (6)$$

where $\text{CGC}(\cdot)$ denotes the dependency graph construction procedure.

4.1.3 Hypothesis Generation

Based on the causal-inspired dependency graph $\mathcal{G}$ and the candidate minor deviation set $\mathcal{T}^e$, GCA performs a global analysis to propose an initial hypothesis of the decisive error of the system failure. The hypothesis is produced by the hypothesis generation process $\text{HG}(\cdot)$ on a compact evidence bundle that includes: the ground truth outcome $\hat{\mathcal{Y}}$, the entire system trace $\mathcal{T}$, the candidate minor deviations $\mathcal{T}^e$, the corresponding textual reasons $\mathcal{R}^e$ to $\mathcal{T}^e$, and the constructed graph $\mathcal{G}$. Formally, the hypothesis generation is formulated as follows:

$$(\tau^*, r^*) = \text{HG}(\hat{\mathcal{Y}}, \mathcal{T}, \mathcal{T}^e, \mathcal{R}^e, \mathcal{G}), \quad (7)$$

where the output τ^* is the hypothesis of the decisive error, and r^* is the corresponding reason.

4.2 Local Counterfactual-inspired Enhancement Module

Practical system traces are often long and dense (Zhang et al., 2025), which exacerbates performance degradation in failure attribution based on full traces. To address this limitation, we introduce the LCE module, which refines τ^* by focusing on a compact local dependency neighborhood. LCE performs a bidirectional search over the causal-inspired dependency graph and uses counterfactual-inspired evaluation with LLM-mediated approximate interventions to assess how correcting candidate interactions may alter the final outcome. By iteratively evaluating local outcome changes, LCE identifies the interaction that most plausibly constitutes the decisive error.

4.2.1 Bidirectional Dependency Neighborhood Search

The bidirectional search aims to identify a minimal set of interactions that provide relevant evidence for explaining the system failure and to localize

the decisive error by tracing dependency propagation across upstream and downstream interactions. Starting from the index i^* of the global hypothesis τ^* , LCE initializes the pivot as $p^{(0)} = i^*$ and explores the causal-inspired dependency graph $\mathcal{G}$ in both forward and backward directions.

The forward search traces downstream dependencies to capture how the hypothesized error may propagate, while the backward search examines upstream dependencies to identify prior misreasoning or missing premises. In both directions, interactions are selected greedily based on counterfactual-inspired evaluation, which assesses how modifying a candidate interaction may affect the final outcome. The union of the forward and backward results forms a locally refined dependency subchain centered on τ^* .

Counterfactual-inspired Evaluation Central to LCE is a counterfactual-inspired evaluation function that assesses the potential causal importance of an interaction by measuring the change in outcome alignment when that interaction is hypothetically corrected instead.

We first construct a fully corrected trace by applying a correction function $\text{Correct}(\cdot)$ conditioned on the ground-truth outcome $\hat{Y}$, the original trace $\mathcal{T}$, extracted erroneous interactions $\mathcal{T}^e$, their corresponding reasons $\mathcal{R}^e$, and the causal-inspired dependency graph $\mathcal{G}$:

$$\hat{\mathcal{T}} = \text{Correct}(\hat{Y}, \mathcal{T}, \mathcal{T}^e, \mathcal{R}^e, \mathcal{G}), \quad (8)$$

where $\hat{\mathcal{T}} = \{\hat{\tau}_1, \dots, \hat{\tau}_N\}$ and $\hat{\tau}_i$ denotes the corrected version of τ_i .

Using $\hat{\mathcal{T}}$, we construct a counterfactual-inspired trace for each interaction τ_i by combining the corrected prefix up to step i with the original suffix from step $i+1$, which is denoted as $\tilde{\mathcal{T}}^{(i)} = \{\hat{\tau}_1, \dots, \hat{\tau}_i, \tau_{i+1}, \dots, \tau_N\}$. This represents a scenario in which only the first i interactions have been corrected. The counterfactual-inspired trace is then evaluated via a simulation process:

$$Y^{(i)} = \text{Simulation}(\tilde{\mathcal{T}}^{(i)}). \quad (9)$$

We measure the semantic alignment between the outcome $Y^{(i)}$ and the ground truth $\hat{Y}$ using a pretrained encoder $\text{Enc}(\cdot)$ and cosine similarity:

$$\mathcal{S}(\tau_i) = \text{Cos}(\text{Enc}(Y^{(i)}), \text{Enc}(\hat{Y})). \quad (10)$$

The marginal attributional impact of correcting τ_i is defined as

$$\Delta\mathcal{S}(\tau_i) = \mathcal{S}(\tau_i) - \mathcal{S}(\tau_{i-1}), \quad (11)$$

which measures the incremental improvement in outcome alignment obtained by correcting the i -th interaction after all preceding interactions have been corrected.

Bidirectional Greedy Search Using the counterfactual-inspired scores, LCE performs a greedy bidirectional search over the causal-inspired dependency graph $\mathcal{G}$.

In the forward search, given the pivot $p^{(l-1)}$ at iteration l , we collect all direct successors denoted as $\mathcal{T}_{\text{eff}}^{(l-1)} = \{\tau_j \mid (p^{(l-1)}, j) \in \mathcal{E}\}$, where $\mathcal{E}$ denotes the edge set of $\mathcal{G}$. The successor with the largest positive marginal contribution is selected:

$$\tau_{j^{(l)}} = \arg \max_{\tau_j \in \mathcal{T}_{\text{eff}}^{(l-1)}} \Delta\mathcal{S}(\tau_j). \quad (12)$$

If $\Delta\mathcal{S}(\tau_{j^{(l)}}) > 0$, the forward candidate set, denoted as $\mathcal{C}_{\text{fwd}}$, is expanded and the pivot updated as $j^{(l)}$. Otherwise, the forward search terminates.

Symmetrically, the backward search traces incoming edges to identify upstream causes denoted as $\mathcal{T}_{\text{cau}}^{(l-1)} = \{\tau_k \mid (k, p^{(l-1)}) \in \mathcal{E}\}$. The predecessor with the strongest positive effect is selected:

$$\tau_{k^{(l)}} = \arg \max_{\tau_k \in \mathcal{T}_{\text{cau}}^{(l-1)}} \Delta\mathcal{S}(\tau_k). \quad (13)$$

If $\Delta\mathcal{S}(\tau_{k^{(l)}}) > 0$, the backward candidate set, denoted as $\mathcal{C}_{\text{bwd}}$, is expanded and the pivot updated as $k^{(l)}$. This process continues until no further positive contributions are found.

These two searches yield a local dependency neighborhood that captures both antecedent causes and propagated effects surrounding τ^* .

4.2.2 Decisive Error Decision

The forward and backward candidate sets are merged with the global hypothesis to form the final local candidate set $\mathcal{T}^c = \{\tau_i \mid i \in \{i^*\} \cup \mathcal{C}_{\text{fwd}} \cup \mathcal{C}_{\text{bwd}}\}$. Each element in $\mathcal{T}^c$ corresponds to an interaction whose counterfactual-inspired correction yields a positive improvement in outcome alignment.

Unlike GCA, which analyzes the full trace, LCE operates only on the interaction subset $\mathcal{T}^c$, avoiding context degradation. This evaluation is formalized by a decisive-error decision function:

$$\tau^* = \text{DED}(\hat{Y}, \mathcal{T}^c, \mathcal{G}), \quad (14)$$

where τ^* denotes the final identified decisive error.

5 Experimental Design

5.1 Dataset

We evaluate DCFA on the Who&When benchmark (Zhang et al., 2025), which is currently the only widely adopted benchmark specifically designed for failure attribution in LLM-based multi-agent systems and serves as the common evaluation testbed for existing baselines (West et al., 2025; Banerjee et al., 2025). The dataset contains 184 multi-agent system traces spanning both Algorithm-generated and Hand-crafted settings. Specifically, 126 traces are generated using CaptainAgent-based systems (Wu et al., 2023), and 58 traces are manually curated from systems such as Magnetic-One (Fourney et al., 2024). The tasks cover web-navigation and general reasoning scenarios derived from AssistantBench (Yoran et al., 2024) and GAIA (Mialon et al., 2023). Each trace is annotated with fine-grained failure labels, including the decisive error and a natural-language explanation. Details are provided in Appendix A.

5.2 Baselines

We compare DCFA against five representative baselines that reflect different LLM-based failure attribution paradigms: All-at-Once, Step-by-Step, and Binary-Search from the Who&When benchmark (Zhang et al., 2025), as well as A2P (West et al., 2025) and ECHO (Banerjee et al., 2025). All methods rely on LLMs for attribution and are evaluated under identical inference settings. We test both open-source and commercial models, including Qwen3-Coder-30B, Qwen3-235B, DeepSeek-R1-32B, DeepSeek-R1-671B, GPT-5, and Gemini-2.5-pro. Details are deferred to Appendix B.

5.3 Implementation Details

In DCFA, the GCA stage is performed by commercial LLMs using the same configurations as the baselines. The LCE stage is executed with a locally deployed Qwen-Coder-30B model. The judgment function J used for dependency condition evaluation in GCA is implemented via structured prompting, where the LLM is queried to perform binary judgments. All LLMs are run with the temperature set to 0 to ensure deterministic inference. All baselines follow identical preprocessing pipelines and prompt configurations as specified in their original papers and released codebases to ensure fair comparison. As ECHO does not provide an official

implementation, we reimplement it based on the descriptions in its paper.

We evaluate attribution quality using Step-level Accuracy, which directly measures precise failure localization and avoids the inflation effects of agent-level metrics. Additional implementation and evaluation details are provided in Appendix C.

5.4 Results and Analysis

5.4.1 Overall Performance

Table 1 reports step-level accuracy of DCFA and five baselines on the Algorithm-generated and Hand-crafted datasets. Compared with the strongest baseline on each dataset, DCFA achieves average improvements of 12.51% and 4.02%, respectively. On the Algorithm-generated dataset, DeepSeek-R1 achieves the highest accuracy, likely benefiting from reasoning-oriented training that aligns with DCFA’s structured causal-inspired dependency graph construction (Guo et al., 2025), while GPT remains competitive on longer traces due to its strong long-context reasoning (Leon, 2025). In contrast, most baselines rely primarily on prompt-induced implicit reasoning, making their attributions less sensitive to the latent dependency relations in multi-agent interactions. For instance, ECHO focuses on observable errors such as tool failures or formatting issues. While these explicit patterns can improve detection accuracy, the resulting deviations are often minor rather than decisive. By tracing such surface symptoms back to their underlying causes, DCFA enables more precise and causally grounded failure attribution. Examples are presented in the case study (Sec. 5.4.4), with additional analysis of GCA-induced causal-inspired dependency graphs in Appendix F.

5.4.2 Ablation on LCE

Since LCE operates on the causal-inspired dependency graph and the hypothesis of the decisive error produced by GCA, we compare GCA-only with DCFA. Table 2 reports an ablation study of DCFA with and without LCE. Adding LCE consistently improves step-level accuracy on both datasets, with 3.45% average improvement on the Hand-crafted dataset and 1.93% average improvement on the Algorithm-generated dataset.

Notably, GCA alone exhibits a noticeable performance drop on the Hand-crafted dataset, where traces are substantially longer (average length = 51.6) than those in the Algorithm-generated dataset (average length = 8.7). In contrast, LCE remains

Model	Algorithm-generated						Hand-crafted					
	All-at-Once	Step-by-Step	Binary-Search	A2P	ECHO	DCFA	All-at-Once	Step-by-Step	Binary-Search	A2P	ECHO	DCFA
Qwen3-Coder-30B	11.90	20.63	13.49	12.70	<u>23.02</u>	37.30	1.72	10.34	6.67	8.62	<u>13.79</u>	15.52
DeepSeek-R1-32B	22.22	29.37	30.16	19.84	<u>30.95</u>	42.06	5.17	15.52	6.90	6.90	<u>18.97</u>	20.69
Qwen3-235B	30.95	25.40	21.43	29.37	<u>36.50</u>	43.65	3.45	15.52	12.07	3.45	<u>17.24</u>	22.41
DeepSeek-R1-671B	16.67	26.98	32.54	15.87	<u>40.48</u>	53.17	3.45	12.07	5.17	6.90	<u>20.69</u>	22.41
GPT-5	15.87	27.78	<u>31.75</u>	15.87	<u>31.75</u>	47.62	3.45	13.79	10.34	13.79	<u>15.52</u>	24.14
Gemini-2.5-pro	26.98	<u>34.13</u>	25.40	32.54	30.95	46.03	5.17	<u>15.52</u>	6.90	10.34	13.79	18.97
Average	20.77	27.72	25.13	21.03	<u>32.28</u>	44.79	3.74	13.46	8.33	8.33	<u>16.67</u>	20.69

Table 1: Step-level Accuracy Comparison on Algorithm-generated and Hand-crafted datasets.

Model	Algorithm-generated		Hand-crafted	
	w/o LCE	DCFA	w/o LCE	DCFA
Qwen3-Coder-30B	33.33	37.30	12.07	15.52
DeepSeek-R1-32B	40.48	42.06	15.52	20.69
Qwen3-235B	42.06	43.65	20.69	22.41
DeepSeek-R1-671B	53.17	53.17	18.97	22.41
GPT-5	44.44	47.62	22.41	24.14
Gemini-2.5-pro	43.65	46.03	13.79	18.97

Table 2: Ablation study of the LCE module.

effective even when global attribution degrades under long-context conditions. This robustness arises because LCE identifies the local dependency chains with the greatest corrective impact through bidirectional search and counterfactual-inspired evaluation, capturing both upstream causes and downstream effects of the error localized by GCA. By focusing the LLM on higher-level and more specific causal-inspired structures, LCE mitigates context degradation and improves attribution precision.

5.4.3 Performance on Varying Trace Lengths

We evaluate DCFA on the Hand-crafted dataset using DeepSeek-R1-671B and 32B, comparing it with all baselines across five context-length levels defined by the Who&When benchmark (Zhang et al., 2025).

As shown in Fig. 3a and Fig. 3b, DCFA outperforms all baselines across nearly all levels, with the largest gains observed at Level 1 and maintained through Level 5. One exception occurs at Level 2 with DeepSeek-R1-32B, where DCFA slightly underperforms ECHO. This may be due to the reduced reliability of weaker LLMs in following the structured dependency reasoning required by DCFA, which can introduce additional variance in LLM-mediated evaluation. Nevertheless, DCFA remains robust overall, achieving strong performance across varying trace lengths.

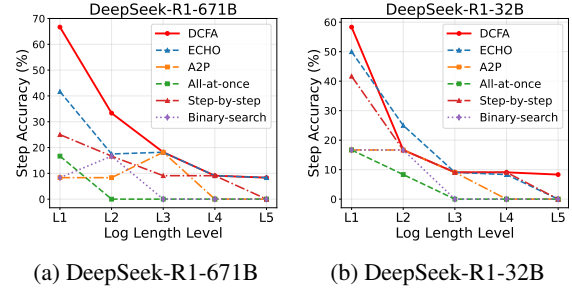


Figure 3: Step-level accuracy of DCFA and baselines across trace lengths using DeepSeek-R1 series LLMs.

5.4.4 Case Study on Causal-inspired Dependency Graph Search

We analyze a system trace from a hand-crafted multi-agent system (ID: 49.json), where three agents (Orchestrator, WebSurfer, and Assistant) collaboratively solve an Unlambda debugging query (see details in Appendix H). The execution trace $\mathcal{T} = \{\tau_1, \dots, \tau_{15}\}$ results in an incorrect prediction of “k”, whereas the correct missing character is the backtick “`”.

Inference Phase of GCA. GCA first identifies multiple erroneous or misleading interactions that are potentially involved in the failure. Specifically, it detects a minor deviation at τ_8 , where WebSurfer returns incomplete operator definitions, and further identifies τ_9 , where the Orchestrator propagates the partial information without further validation. These errors form a dependency chain and jointly contribute to the subsequent failure, rather than constituting independent failure sources. Based on the resulting causal-inspired dependency graph, GCA localizes the relevant error region for further refinement, as shown in Figure 4.

Refinement Phase via LCE. LCE then conducts localized, bidirectional counterfactual-inspired reasoning over the error region identified by GCA. By examining the corrective effects of candidate interactions along the dependency graph, LCE deter-

mines that τ_{12} has the greatest impact on mitigating the system failure. At τ_{12} , the Assistant fabricates a solution based on incomplete semantics, which further propagates the accumulated errors toward the incorrect final outcome. Although correcting τ_8 or τ_9 can mitigate the error propagation, neither correction is sufficient to fully resolve the failure. In contrast, correcting τ_{12} provides the largest reduction in the failure, thereby identifying it as the decisive error. The refinement process is illustrated in Figure 4.

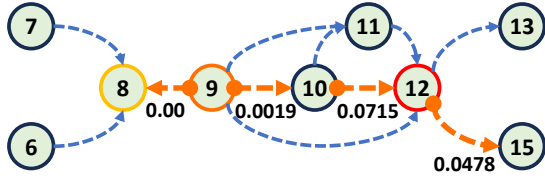


Figure 4: LCE refinement on the causal-inspired dependency graph of trace 49.json. Orange dashed arrows indicate search paths. Forward and backward traversal evaluates the corrective effects of candidate interactions, isolating τ_{12} as the decisive error. Yellow: minor deviation (τ_8); Orange: GCA-identified error (τ_9); Red: decisive error (τ_{12}).

Decisive Error and DCFA Attribution. This case exemplifies the multi-error setting in our problem formulation, where multiple errors jointly contribute to the system failure while no single correction among the earlier errors fully recovers the correct outcome. Specifically, τ_8 and τ_9 contribute to the error propagation and can be corrected to mitigate the failure, whereas τ_{12} has the greatest corrective effect and is therefore identified as the decisive error. By combining global causal-inspired dependency search with local counterfactual-inspired refinement, DCFA successfully distinguishes contributing errors from the decisive error and accurately attributes the system failure to τ_{12} . Additional details are provided in Appendix H.

5.4.5 Computational Cost

We analyze DCFA’s computational cost by decomposing it into GCA and LCE. GCA relies on commercial LLM API calls, while LCE is executed locally and adds only inference-time overhead. Following (Zhang et al., 2025), output-token costs are ignored unless stated otherwise.

GCA performs two full-context LLM calls: deviation detection with dependency graph construction, and decisive-error refinement. Since outputs are small relative to the input context, the total cost

Method	Runtime (s)	Tokens (k)	Accuracy
All-at-Once	15	6	12.26
Search-by-Step	61	9	20.59
Binary-Search	25	12	16.73
A2P	15	6	14.68
ECHO	31	12	24.48
DCFA w/o LCE	32	13	<u>30.05</u>
DCFA	300	32	32.74

Table 3: Approximate runtime, token usage, and attribution accuracy on the Who&When benchmark.

can be approximated as

$$\text{Cost}_{\text{GCA}} \approx 2C + 2\bar{n}L, \quad (15)$$

where C is the prompt overhead, $\bar{n}$ the average trace length, and L the token length per step. Notably, the strongest baseline ECHO requires the same number of full-context calls, resulting in identical computational complexity, while GCA alone already yields substantial accuracy improvements.

The LCE module runs locally and introduces additional inference overhead. Its complexity can be approximated as

$$\text{Cost}_{\text{LCE}} \approx mC + m\bar{n}L, \quad (16)$$

where $m \approx 4.5$ is the average number of bidirectional neighborhood expansions. Although LCE increases runtime, it further improves attribution accuracy, especially on longer trajectories.

Table 3 summarizes the approximate runtime, token consumption, and attribution accuracy of all compared methods on the Who&When benchmark. DCFA without LCE incurs a cost comparable to ECHO while achieving substantially higher accuracy (30.05 vs. 24.48). Adding LCE increases the runtime from 32s to 300s and token consumption from 13k to 32k, but further improves accuracy to 32.74. Detailed runtime statistics, token usage, and comprehensive comparisons with baseline methods are provided in Appendix E.

6 Conclusion

In this work, we propose DCFA, a training-free framework for failure attribution in LLM-based multi-agent systems. Combining global causal-inspired dependency graph analysis with local counterfactual-inspired reasoning, DCFA identifies decisive errors and performs robustly across traces of varying lengths. Experiments show it improves step-level attribution accuracy by up to 8.27% over state-of-the-art baselines and remains effective on challenging long-context traces.

Limitations

Dependence on LLM Capability. DCFA relies on the reasoning inference capabilities of the underlying large language models. When system traces become very long, LLMs may struggle to maintain coherent dependency representations, which can reduce the accuracy of global causal-inspired dependency graph construction. Similarly, when using smaller or less capable models for local counterfactual-inspired refinement, the precision of decisive error identification may be constrained.

Scalability to Long or Complex Traces. Although the global-local combination in DCFA improves robustness to moderately long execution traces, extremely long or highly branching multi-agent interactions can still pose challenges. In such cases, both global causal-inspired attribution and local counterfactual-inspired reasoning may become less reliable, limiting applicability to systems with deeply nested or prolonged interaction patterns.

Attribution of Intrinsic LLM Reasoning Errors. Compared with the decisive errors that reflect failures within the MAS workflow. Errors originating from inherent LLM hallucinations may be difficult to attribute accurately.

Ethical Considerations

This research is intended as a diagnostic tool to support failure analysis in LLM-based multi-agent systems, rather than an automated mechanism for judgment or accountability. Its attribution results depend on the reasoning behavior of underlying language models and may be imperfect, especially when failures stem from intrinsic model errors or ambiguous interactions. Over-reliance on such automated explanations could lead to misinterpretation of responsibility if used without human oversight. Therefore, we highlight that the proposed DCFA should be applied with caution in high-stakes settings and used to assist, not replace, human analysis of system failures.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grants 62572346 and 62322208.

References

- Taiyu Ban, Lyuzhou Chen, Derui Lyu, Xiangyu Wang, Qinrui Zhu, Qiang Tu, and Huanhuan Chen. 2025. Integrating large language model for improved causal discovery. *IEEE Transactions on Artificial Intelligence*.
- Adi Banerjee, Anirudh Nair, and Tarik Borogovac. 2025. Where did it all go wrong? a hierarchical look into multi-agent error attribution. In *NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling*.
- Zoltán Barta, Balázs Nagy, and László Gulyás. 2025. Measuring the robustness of multi-agent reinforcement learning systems under partial agent failure. In *Proceedings of the Intelligent Robotics FAIR 2025*, pages 58–63.
- Marco Becattini, Roberto Verdecchia, and Enrico Vicario. 2025. Sallma: A software architecture for llm-based multi-agent systems. In *2025 IEEE/ACM International Workshop New Trends in Software Architecture (SATrends)*, pages 5–8. IEEE.
- Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. 2026. Why do multi-agent llm systems fail? *Advances in Neural Information Processing Systems*, 38.
- Qing Cheng, Zefan Zeng, Xingchen Hu, Yuehang Si, and Zhong Liu. 2025. A survey of event causality identification: Taxonomy, challenges, assessment, and prospects. *ACM Computing Surveys*, 58(3):1–37.
- Kai-Hendrik Cohrs, Emiliano Diaz, Vasileios Sitokostas, Gherardo Varando, and Gustau Camps-Valls. 2025. Large language models for causal hypothesis generation in science. *Machine Learning: Science and Technology*, 6(1):013001.
- Victor-Alexandru Darvari, Stephen Hailes, and Mirco Musolesi. 2024. Large language models are effective priors for causal graph discovery. *arXiv preprint arXiv:2405.13551*.
- Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. 2025. Trail: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*.
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, et al. 2024. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*.
- Lisa R Goldberg. 2019. The book of why: The new science of cause and effect: by judea pearl and dana mackenzie, basic books (2018). isbn: 978-0465097609.

- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Lewis Hammond, Alan Chan, Jesse Clifton, Jason Hoelscher-Obermaier, Akbir Khan, Euan McLean, Chandler Smith, Wolfram Barfuss, Jakob Foerster, Tomáš Gavenčiak, et al. 2025. Multi-agent risks from advanced ai. *arXiv preprint arXiv:2502.14143*.
- Xu He, Di Wu, Yan Zhai, and Kun Sun. 2025. Sentinelagent: Graph-based anomaly detection in multi-agent systems. *arXiv preprint arXiv:2505.24201*.
- Yi Huang, Fangyin Cheng, Fan Zhou, Jiahui Li, Jian Gong, Hongjun Yang, Zhidong Fan, Caigao Jiang, Siqiao Xue, and Faqiang Chen. 2024. Romas: A role-based multi-agent system for database monitoring and planning. *arXiv preprint arXiv:2412.13520*.
- Kosuke Imai and Kentaro Nakamura. 2026. Causal inference with generative artificial intelligence: Application to texts as treatments. *Journal of the American Statistical Association*, (just-accepted):1–27.
- Thomas Jiralerspong, Xiaoyin Chen, Yash More, Vedant Shah, and Yoshua Bengio. Efficient causal graph discovery using large language models. In *ICLR 2024 Workshop: How Far Are We From AGI*.
- Maikel Leon. 2025. Gpt-5 and open-weight large language models: Advances in reasoning, transparency, and control. *Information Systems*, page 102620.
- Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinityearth*, 1(1):9.
- Yanhui Li, Linghan Meng, Lin Chen, Li Yu, Di Wu, Yuming Zhou, and Baowen Xu. 2022. Training data debugging for the fairness of machine learning software. In *Proceedings of the 44th International Conference on Software Engineering*, pages 2215–2227.
- Cheng Liu, Wei Xiang, and Bang Wang. 2024a. Identifying while learning for document event causality identification. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3815–3827.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024b. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046.
- Xiaoyu Liu, Paiheng Xu, Junda Wu, Jiaxin Yuan, Yifan Yang, Yuhang Zhou, Fuxiao Liu, Tianrui Guan, Haoliang Wang, Tong Yu, et al. 2025. Large language models and causal inference in collaboration: A comprehensive survey. *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 7668–7684.
- Kun Luo, Tong Zhou, Yubo Chen, Jun Zhao, and Kang Liu. 2024. Open event causality extraction by the assistance of llm in task annotation, dataset, and method. In *Proceedings of the Workshop: Bridging Neurons and Symbols for Natural Language Processing and Knowledge Graphs Reasoning (Neusym-Bridge)@ LREC-COLING-2024*, pages 33–44.
- Yacine Majdoub, Eya Ben Charrada, and Haifa Touati. 2025. Towards adaptive software agents for debugging. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pages 636–640.
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*.
- Yu Pan, Jianxin Sun, Hongfeng Yu, Joe Luck, Geng Bai, Nipuna Chamara, Yufeng Ge, and Tala Awada. 2024. Building multi-agent copilot towards autonomous agricultural data management and analysis. In *2024 IEEE International Conference on Big Data (Big-Data)*, pages 4384–4393. IEEE.
- Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, et al. 2025. Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 422–431.
- Krishna Ronanki. 2025. Facilitating trustworthy human-agent collaboration in llm-based multi-agent system oriented software engineering. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pages 1333–1337.
- Xu Shen, Qi Zhang, Song Wang, Zhen Tan, Xinyu Zhao, Laura Yao, Vaishnav Tadiparthi, Hossein Nourkhiz Mahjoub, Ehsan Moradi Pari, Kwonjoon Lee, et al. 2026. Metacognitive self-correction for multi-agent system via prototype-guided next-execution reconstruction. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 23320–23337.
- Shobhit Shrotriya, N Banu, Avi Kulkarni, and Sujata Aiyangar. 2025. Navigating the path to ethical & responsible ai integration in health & life sciences with human and machine collaboration. *Journal of the Society for Clinical Data Management*, 5(1):1–12.
- Ya Su, Hu Zhang, Guangjun Zhang, Yujie Wang, Yue Fan, Ru Li, and Yuanlong Wang. 2025. Enhancing event causality identification with llm knowledge and concept-level event relations. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 7403–7414.
- Chuanneng Sun, Songjun Huang, and Dario Pompili. 2025. Llm-based multi-agent decision-making: Challenges and future directions. *IEEE Robotics and Automation Letters*.

- Song Tong, Kai Mao, Zhen Huang, Yukun Zhao, and Kaiping Peng. 2024. Automating psychological hypothesis generation with ai: when large language models meet causal graph. *Humanities and Social Sciences Communications*, 11(1):896.
- Martin Tutek, Fateme Hashemi Chaleshtori, Ana Marasović, and Yonatan Belinkov. 2025. Measuring chain of thought faithfulness by unlearning reasoning steps. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9946–9971.
- Vishal Verma, Sawal Acharya, Samuel Simko, Devansh Bhardwaj, Anahita Haghighat, Dominik Janzing, Mrinmaya Sachan, Zhijing Jin, and Yongjin Yang. 2025. Causal ai scientist: Facilitating causal data science with large language models. In *NeurIPS 2025 AI for Science Workshop*.
- Haoyu Wang, Fengze Liu, Jiayao Zhang, Dan Roth, and Kyle Richardson. 2024. Event causality identification with synthetic control. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1725–1737.
- Alva West, Yixuan Weng, Minjun Zhu, Zhen Lin, Zhiyuan Ning, and Yue Zhang. 2025. Abduct, act, predict: Scaffolding causal inference for automated failure attribution in multi-agent systems. *arXiv preprint arXiv:2509.10401*.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying llm-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824.
- Bingyu Yan, Zhibo Zhou, Litian Zhang, Lian Zhang, Ziyi Zhou, Dezhuang Miao, Zhoujun Li, Chaozhao Li, and Xiaoming Zhang. 2025. Beyond self-talk: A communication-centric survey of llm-based multi-agent systems. *arXiv preprint arXiv:2502.14321*.
- Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. 2026. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems. *Advances in Neural Information Processing Systems*, 38:107309–107336.
- Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. Assistantbench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8938–8968.
- Longxuan Yu, Delin Chen, Siheng Xiong, Qingyang Wu, Dawei Li, Zhikai Chen, Xiaoze Liu, and Liangming Pan. 2025. Causaleval: Towards better causal reasoning in language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 12512–12540.
- Zefan Zeng, Qing Cheng, Xingchen Hu, Wentao Li, Weiping Ding, and Zhong Liu. 2026. Zero-shot event causality identification via multisource evidence fuzzy aggregation with large language models. *IEEE Transactions on Fuzzy Systems*.
- Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026. Agentracer: Who is inducing failure in the llm agentic systems? In *International Conference on Learning Representations*, volume 2026, pages 11377–11399.
- Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 76583–76599. PMLR.
- Jingwen Zhou, Jieshan Chen, Qinghua Lu, Dehai Zhao, and Liming Zhu. 2025. Shielda: Structured handling of exceptions in llm-driven agentic workflows. *arXiv preprint arXiv:2508.07935*.
- Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, et al. 2025. Where llm agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.
- Yaochen Zhu, Yinhan He, Jing Ma, Mengxuan Hu, Sheng Li, and Jundong Li. 2024. Causal inference with latent variables: Recent advances and future perspectives. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 6677–6687.

A Dataset

We utilize the datasets from the Who&When benchmark (Zhang et al., 2025), which comprise both Algorithm-generated and Hand-crafted multi-agent system traces, totaling 184 executions. Specifically, 126 traces are generated algorithmically using multi-agent systems built on CaptainAgent (Wu et al., 2023), while 58 traces are manually curated from Hand-crafted systems such as Magnetic-One (Fourney et al., 2024). The dataset encompasses a wide range of realistic multi-agent scenarios based on queries from GAIA (Mialon et al., 2023) and AssistantBench (Yoran et al., 2024).

The dataset covers a broad set of multi-agent tasks, including web-navigation-style challenges derived from AssistantBench (Yoran et al., 2024) and general-assistant reasoning tasks inspired by GAIA (Mialon et al., 2023). Each trace is annotated with fine-grained failure information, including the responsible agent, the true decisive error, and a natural language explanation of the failure.

The two categories exhibit distinct characteristics in terms of agent participation and trace length. Algorithm-generated traces involve less than 4 agents, with trace lengths ranging from 5 to 10 steps, a mean length of 8.7 steps, and a median length of 10 steps. In contrast, Hand-crafted traces involve between 1 and 5 agents, with trace lengths spanning 5 to 130 steps, a mean length of 51.6 steps, and a median length of 32 steps. The distributions of both datasets, illustrated in Fig. 5, highlight the greater variability present in the Hand-crafted traces.

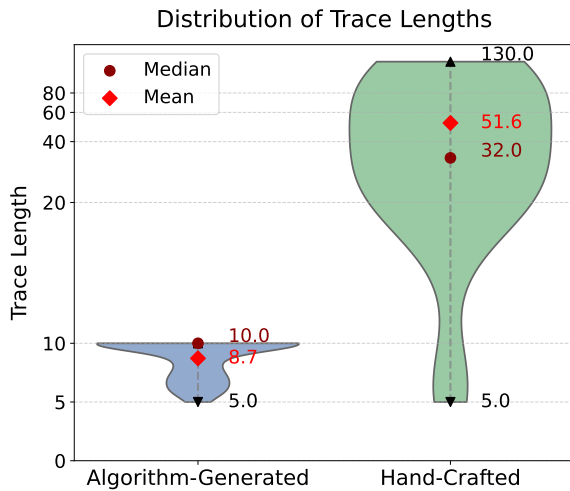


Figure 5: Violin plot showing the distribution of trace lengths for the Algorithm-generated and Hand-crafted datasets. The width of each violin represents the kernel density estimation of the data distribution, illustrating the frequency of different trace lengths. The central red line indicates the median, while the black triangles mark the minimum and maximum values. Notably, wider regions denote higher concentrations of trace lengths within that range.

Moreover, some benchmarks, such as AgentErrorBench with AgentDebug (Zhu et al., 2025) and TRAIL (Deshpande et al., 2025), may appear superficially similar to our task setting. However they differ fundamentally from our problem formulation and evaluation objective, making direct comparison inappropriate. AgentErrorBench focuses on human-agent interaction and single-agent trajec-

tries, where the objective is to detect the earliest mistake that initiates cascading reasoning distortions leading to failure. TRAIL is designed for identifying and analyzing all errors in trajectories. In contrast, this work focuses on failure attribution in LLM-based multi-agent systems, where the decisive error is defined as the earliest mistake whose correction could reverse the overall system failure. For example, an upstream deviation (an agent returning incomplete search results) eventually leads to a downstream decisive error (another agent fabricating missing information). AgentErrorBench would typically detect the earlier deviation, which could potentially be mitigated through subsequent agent interactions such as verification or additional tool usage, whereas this work aims to identify the interaction whose counterfactual-inspired correction would directly rescue the final outcome.

B Baselines

To evaluate the effectiveness of DCFA, we compare it against five representative baseline strategies that differ in how the LLM interacts with the failure traces: All-at-Once, Step-by-Step, and Binary-Search from the Who&When benchmark (Zhang et al., 2025), as well as A2P (West et al., 2025) and ECHO (Banerjee et al., 2025). Each baseline reflects a distinct reasoning paradigm in localizing the failure-responsible step and agent within multi-agent system traces.

- **All-at-Once.** The algorithm leverages the LLM to interpret the entire system trace in a single pass and directly infer the decisive error of the system failure.
- **Step-by-Step.** The algorithm partitions the complete system trace into individual interactions. At each step, the model determines whether an error has occurred in the current interaction. If an error is detected, the process terminates immediately and the LLM outputs the corresponding step index. Otherwise, the reasoning continues sequentially until the final interaction is reached.
- **Binary-Search.** The algorithm begins with the query and the full failure log. It first determines whether the fault occurred in the upper or lower half of the trace. The identified half is then provided for further inspection. This iterative procedure continues until a single interaction is isolated as the decisive error.
- **A2P.** The algorithm employs a causal-inspired

scaffold that sequentially guides LLMs through abductive hypothesis generation, explicit action-level intervention, and short-horizon counterfactual-inspired simulation to assess whether correcting a given step can reverse a system failure.

- **ECHO.** The algorithm introduces a hierarchical error attribution approach for multi-agent systems, explicitly targeting both minor execution deviations and overt errors within long interaction traces. Through multi-level contextual abstraction and consensus-based analysis, ECHO can detect early, subtle deviations that precede downstream failures, enabling fine-grained and reliable error localization.

All the baselines are based on the LLMs to achieve the failure attribution. To comprehensively assess each algorithm, we test them with various LLMs, including both open-source models of different sizes and commercial models. The models tested include the Qwen3 series (Qwen3-Coder-30B and Qwen3-235B), the DeepSeek series (DeepSeek-R1-32B and DeepSeek-R1-671B), GPT-5, and Gemini-2.5-pro. All baselines are evaluated under identical inference conditions to ensure a fair and consistent comparison.

C Implementation Details

Our approach operates in a training-free manner. In the GCA stage, minor deviation identification, causal-inspired dependency graph construction, and hypothesis generation are performed by the selected commercial LLM, following the same configuration as the baseline setup. In the LCE stage, the bidirectional dependency neighborhood search and decisive error decision are executed using the locally deployed Qwen-Coder-30B model. The encoder used to transform the outcomes generated by the simulation process into a continuous embedding space is implemented using the text-embedding-3-large API provided by OpenAI.

The judgment function J is implemented as a prompt-based evaluator operating over multi-agent system traces. Rather than re-executing the entire system, J receives a system trace containing agent identities and message contents during in the interactions, and is prompted to assess whether a target interaction would plausibly occur under a counterfactual-inspired intervention

Commercial LLMs are accessed through their respective APIs, while for local experiments, we de-

ploy Qwen-Coder-30B on a workstation equipped with two NVIDIA A800 GPUs. For both commercial and locally deployed LLMs, the temperature is fixed to 0 to ensure deterministic outputs.

All inference procedures of the baseline methods follow identical preprocessing steps and prompt configurations on the Who&When benchmark to ensure fair and reproducible evaluation across different models.

To evaluate attribution quality, we focus on Step-level Accuracy, which measures the proportion of cases in which the exact erroneous step is correctly identified. Notably, we do not focus on Agent-level Accuracy, as in systems with a small number of agents, Agent-level Accuracy can be inflated. This occurs because it overlooks the more precise Step-level localization of failures. Given our emphasis on accurate localization at both the agent and step levels, we exclusively use Step-level Accuracy, rejecting the misleading Agent-level baseline.

D Robustness Across Independent Runs

Although we set the decoding temperature to 0 for all experiments, minor randomness in LLM inference may still lead to small fluctuations in the outputs. To evaluate the robustness of our results, we repeated the experiments using the three best-performing LLMs and report the mean and standard deviation over three independent runs.

As shown in Table 4, the variance across runs is extremely small for all methods and models. This indicates that the observed performance differences are stable and not caused by stochastic variations in the LLM outputs. In particular, DCFA consistently achieves the highest performance across all settings with minimal variance, demonstrating the robustness of the proposed failure attribution framework.

E Detailed Computational Cost Analysis

E.1 Computational Cost Calculation

This section reports detailed empirical statistics of token usage and runtime for DCFA and baseline methods on the Who&When benchmark. For DCFA, the average prompt overhead is approximately 400 tokens. The average trace length is about 3k tokens for the Algorithm-Generated dataset and 10k tokens for the Hand-Crafted dataset. The GCA stage performs two full-context API calls, each taking roughly 16 seconds, resulting in an average runtime of about 32 seconds per instance.

Method	Algorithm-Generated			Hand-Crafted		
	DeepSeek-R1-671B	GPT-5	Gemini-2.5-pro	DeepSeek-R1-671B	GPT-5	Gemini-2.5-pro
All-at-Once	16.93 $\pm$ 0.00	16.14 $\pm$ 0.00	27.51 $\pm$ 0.02	4.02 $\pm$ 0.01	3.45 $\pm$ 0.00	4.60 $\pm$ 0.01
Step-by-Step	27.25 $\pm$ 0.03	28.31 $\pm$ 0.03	34.92 $\pm$ 0.03	11.50 $\pm$ 0.01	13.22 $\pm$ 0.01	16.10 $\pm$ 0.01
Binary-Search	33.86 $\pm$ 0.01	31.75 $\pm$ 0.03	26.46 $\pm$ 0.02	5.75 $\pm$ 0.01	10.92 $\pm$ 0.01	5.75 $\pm$ 0.01
A2P	16.40 $\pm$ 0.04	16.40 $\pm$ 0.05	31.75 $\pm$ 0.05	6.32 $\pm$ 0.01	12.64 $\pm$ 0.01	9.77 $\pm$ 0.01
ECHO	40.21 $\pm$ 0.00	31.48 $\pm$ 0.03	31.22 $\pm$ 0.00	19.54 $\pm$ 0.09	13.79 $\pm$ 0.01	13.22 $\pm$ 0.08
DCFA	53.44 $\pm$ 0.01	47.09 $\pm$ 0.04	46.69 $\pm$ 0.01	21.38 $\pm$ 0.04	22.41 $\pm$ 0.01	21.84 $\pm$ 0.03

Table 4: Robustness evaluation across three independent runs. We report mean performance and standard deviation. The extremely small variance indicates that the results are stable despite potential randomness in LLM inference.

The LCE stage is executed locally using Qwen3-Coder-30B on two NVIDIA A800 GPUs. Each inference processes approximately 6k tokens and takes about 60 seconds. Since bidirectional dependency neighborhood search requires on average 4.5 such calls, the total LCE runtime is approximately 270 seconds. Consequently, the overall runtime of DCFA is around 300 seconds per trajectory.

Baseline methods differ primarily in how they process the interaction trace. All-at-Once analyzes the entire trace using a single LLM call of roughly 6k tokens. Search-by-Step processes one interaction at a time (about 50 tokens per step) and averages around 20 steps per trace, resulting in approximately 9k tokens. Binary-Search iteratively analyzes halves of the remaining trace and typically requires about five calls, leading to roughly 12k tokens. A2P uses a prompt of about 300 tokens and processes the full trace in a single call, while ECHO also uses a prompt of about 300 tokens but performs two full-context calls.

E.2 Trade-off Analysis

Although DCFA introduces additional computational overhead, it yields substantial performance improvements. In terms of computational complexity, the strongest baseline, ECHO, and the GCA component of DCFA both require two full-context LLM calls. Notably, the GCA module alone already achieves clear improvements over ECHO (+10.58% on the Algorithm-Generated dataset and +0.57% on the Hand-Crafted dataset), suggesting that the primary performance gains stem from enhanced reasoning over the trajectory. The LCE module, in contrast, relies on local LLMs and therefore incurs relatively additional cost. It further improves detection accuracy on longer trajectories (e.g., +3.45% on the Hand-Crafted dataset), indicating a favorable trade-off between attribution accuracy and computational efficiency.

E.3 Runtime Cost and Practicality

DCFA is designed as a post-hoc failure attribution framework, consistent with baselines such as ECHO and A2P, rather than as a real-time debugging module. Its primary goal is to analyze the complete system trace after a task failure and identify the decisive errors. Insights from this analysis can guide offline modifications to the MAS, including agent structures, context management strategies, or tool invocation policies, rather than adjusting the system during execution.

For online or real-time scenarios, a different design is required. Incremental reasoning can be applied at each interaction step, leveraging cached causal-inspired dependency subgraphs and restricting counterfactual-inspired evaluations to local neighborhoods. This avoids repeatedly processing the full trace and can substantially reduce runtime overhead, making failure attribution more practical for long-running or continuously operating multi-agent systems.

F Supplementary Analysis of GCA-Induced Causal-inspired Dependency Graphs

We analyze the structural statistics of the causal-inspired dependency graphs constructed during the GCA stage, with a particular focus on cross-model variability and its implications for decisive error localization. To facilitate a concrete comparison, we further present representative causal-inspired dependency graphs generated by GPT-5 and Gemini-2.5-pro on a selected sample (49.json) from the Hand-crafted dataset.

F.1 Model Capacity and Deviations in Causal-inspired Dependency Graph Edge Density

Table 5 reports the average number of edges per graph across different LLMs on both the Algorithm-

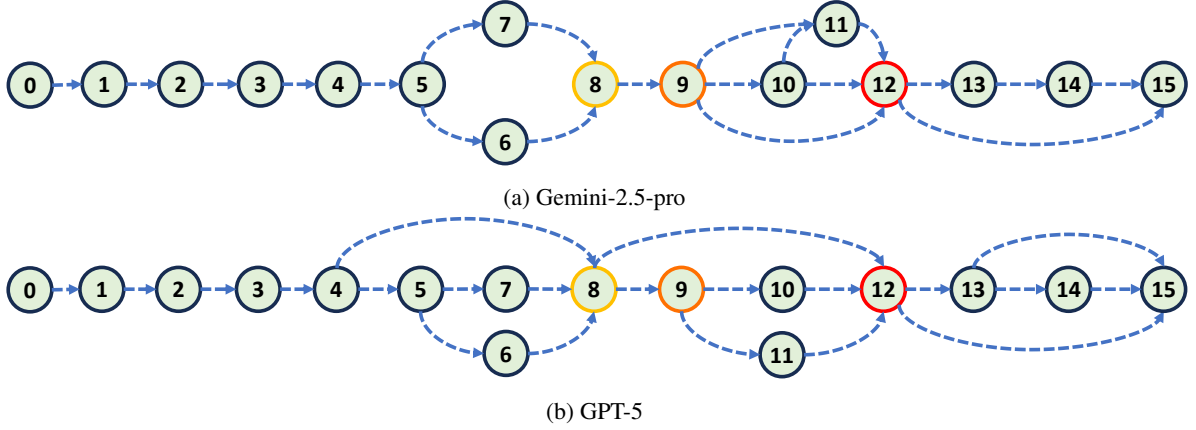


Figure 6: Representative causal-inspired dependency graphs constructed by Gemini-2.5-pro and GPT-5 on the Hand-crafted example 49.json. Differences are most pronounced in the critical failure region $\tau_9-\tau_{12}$.

generated and Hand-crafted datasets.

Model	Algorithm-generated	Hand-crafted
Qwen3-Coder-30B	7.89	68.90
DeepSeek-R1-32B	8.24	46.39
Qwen3-235B	9.41	61.52
DeepSeek-R1-671B	9.25	57.09
GPT-5	10.12	55.40
Gemini-2.5-pro	8.76	56.38
Average	8.95	56.61

Table 5: Average number of edges per causal-inspired dependency graph generated during the GCA stage on the Algorithm-generated and Hand-crafted datasets.

A clear stratification emerges when comparing models of different capability. Less capable models, such as Qwen3-Coder-30B and DeepSeek-R1-32B, exhibit pronounced deviations from the overall mean, especially on the Hand-crafted dataset, producing overly dense or sparse causal-inspired dependency graphs (e.g., 68.90 edges for Qwen3-Coder-30B and 46.39 edges for DeepSeek-R1-32B). Such extreme deviations, in either direction, indicate unstable causal induction and low quality of the causal-inspired dependency graph generation, likely due to context degradation in LLMs.

By contrast, higher-capability models, such as GPT-5, Gemini-2.5-pro, and DeepSeek-R1-671B, maintain edge counts closer to the global average, suggesting a stronger ability to suppress irrelevant dependencies and preserve salient relations.

F.2 Implications for Decisive Error Localization.

To qualitatively assess how these structural differences affect downstream reasoning, Figure 6

visualizes the causal-inspired dependency graphs constructed by Gemini-2.5-Pro and GPT-5 on the same hand-crafted example, 49.json. Although both models successfully capture the overall dependency structure, their local structures in the critical failure region differ substantially.

Specifically, the GCA stage using Gemini-2.5-Pro initially hypothesizes τ_9 as the decisive error, reflecting a more entangled dependency subgraph in the interval $\tau_9-\tau_{12}$. In contrast, GCA with GPT-5 constructs a more compact and hierarchical causal-inspired dependency structure over the same region, enabling it to identify τ_{12} as the decisive error.

Importantly, this discrepancy does not prevent DCFA with Gemini-2.5-Pro from ultimately recovering the correct failure source. Through the LCE module in the subsequent DCFA stage, the model performs counterfactual-inspired reasoning over the extracted local dependency subgraph, progressively evaluating and eliminating less relevant upstream candidates and converging on the decisive error τ_{12} . This observation highlights that while reliable global causal-inspired dependency graphs facilitate earlier localization, the proposed framework remains robust to structural noise through localized counterfactual-inspired refinement.

G Edge Growth and Long-Trace Scalability

Analysis of the causal-inspired dependency graphs constructed by DCFA shows that edge growth is nearly linear with trajectory length. In realistic multi-agent traces, each interaction typically links only to temporally or semantically adjacent steps, rather than to all preceding interactions. For example, in the Algorithm-generated dataset, traces

average 8.7 steps with 8.95 edges, while in the Hand-crafted dataset, traces average 51.6 steps with 56.61 edges. This correspondence indicates that edge growth scales linearly rather than quadratically, mitigating combinatorial complexity.

Maintaining consistent structured reasoning over long traces remains challenging for LLM-based approaches. LCE addresses this challenge by performing localized exploration within the dependency graph, reducing the reasoning scope and keeping the additional search overhead roughly linear in trace length. Alternative cost-efficient strategies could further alleviate scaling issues, including heuristic graph pruning (e.g., limiting search depth or filtering low-confidence edges), sliding-window reasoning without full graph traversal, or hierarchical trace compression prior to graph construction. These approaches offer options for scaling DCFA to longer or more complex trajectories while retaining the benefits of localized dependency reasoning.

H Details for Case Study on Causal-inspired Dependency Graph Search

To demonstrate the effectiveness of DCFA, we present a case study based on a system trace from the Hand-crafted multi-agent system (ID: 49.json). In this scenario, the user query triggers interactions among three agents: Orchestrator, WebSurfer, and Assistant. The Orchestrator decomposes the task and coordinates the reasoning workflow, the WebSurfer retrieves external information, and the Assistant integrates the gathered evidence to produce the final answer.

The system trace captures each step of the multi-agent reasoning process, including information collection, context propagation, and intermediate reasoning actions. Specifically, the detail content of the user query is shown in the following box:

User Query

In Unlambda, what exact character or text should be added to make the given code output “For penguins”? Answer with the name of the needed character if applicable.
Code: `r“.....F.o.r .p.e.n.g.u.i.n.si`

In response, the multi-agent system generates the system trace consisting of fifteen interactions $\mathcal{T} = \{\tau_1, \dots, \tau_{15}\}$. Each represents an interaction from one agent, as illustrated in Fig. 7. The sys-

tem produces an incorrect answer “k”, whereas the correct missing character should be backtick “`”.

Within this trace, some steps involve minor deviations, such as incomplete retrieval or partial context propagation, which are generally correctable through downstream verification. In contrast, a decisive error occurs when the Assistant fabricates information based on incomplete or ambiguous input, producing content that is irreversible and misguides subsequent reasoning, ultimately leading to an incorrect outcome.

This overview sets the stage for a detailed case study analysis, where we examine how DCFA distinguishes contributory minor deviations from the decisive fabrication error and demonstrates its capability to pinpoint the root cause of failure in multi-agent reasoning.

Inference Phase of GCA During global analysis, GCA first detects a potential minor deviation at τ_8 , where the WebSurfer attempts to retrieve definitions of Unlambda operators but returns incomplete information.

Excerpt from τ_8

WebSurfer: Searching for Unlambda operators “.”, “^”, and “r”...

Based on this, GCA builds a causal-inspired dependency graph centered on τ_8 (Fig. 4) and ultimately identifies and attributes the system failure to τ_9 , where the Orchestrator mistakenly assumes that the retrieved information is complete and proceeds without verification.

Excerpt from τ_9

Orchestrator: Progress confirmed. Proceeding to analyze gathered information...

Refinement Phase of LCE. To refine this hypothesis of GCA, the Local Counterfactual-inspired Enhancement (LCE) module focuses on the neighborhood around τ_9 , forming a local subgraph (Fig. 4). It conducts counterfactual-inspired simulations to evaluate whether modifying prior steps would correct the outcome. Forward traversal from τ_9 to τ_8 shows no effect, while backward traversal identifies τ_{12} as the decisive cause.

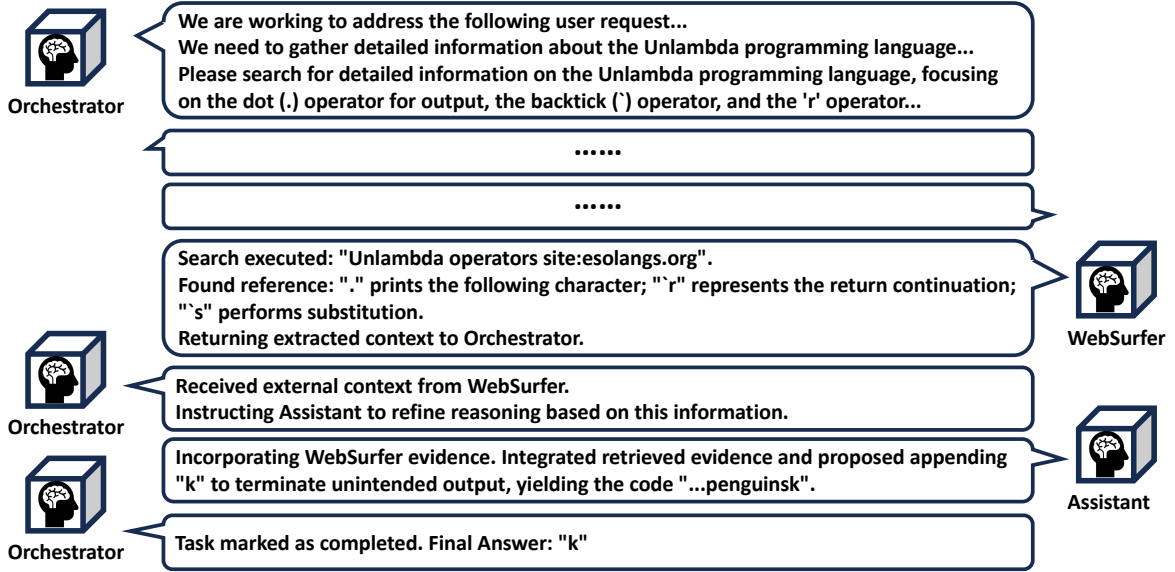


Figure 7: Key interaction steps of incorrect answer generation in trace 49.json.

Stage	Identified Decisive Error	Agent	Supporting Evidence
GCA (Global)	τ_9	Orchestrator	Ignore missing operator details
LCE (Local)	τ_{12}	Assistant	Fabricate missing content

Table 6: Comparison between GCA and LCE Attributions.

Excerpt from τ_{12}

Assistant: Summarizing Unlambda constructs... The backtick (`) applies functions, the dot (.) outputs characters, and the "r" operator reads inputs. Based on this, adding "k" may terminate unwanted outputs.

LCE identifies τ_{12} as the decisive error, where the Assistant fabricates knowledge by falsely claiming that the WebSurfer’s incomplete results included operator definitions. This misinformation misleads subsequent reasoning, ultimately producing the wrong output in τ_{15} .

Decisive Error and DCFA Attribution. Table 6 compares the decisive error attributions identified by the GCA and LCE modules. In this trace, τ_8 corresponds to the WebSurfer attempting to retrieve essential information about Unlambda operators. Although the retrieved information is incomplete, this step constitutes a minor deviation: it is benign and can be effectively corrected by the system’s subsequent verification mechanisms, such as detecting missing fields and re-invoking tools to retrieve the omitted information. Therefore, τ_8 does not constitute a decisive error.

τ_9 represents the Orchestrator proceeding and propagating the partial context returned by WebSurfer. While the GCA module flags τ_9 as a global-level misjudgment, this step primarily propagates the minor deviation from τ_8 rather than introducing a fundamentally new error, making it contributory but not causative.

The decisive error occurs at τ_{12} , where the Assistant integrates the retrieved information and fabricates operator details, transforming a recoverable incompleteness into an irreversible factual error. This fabricated knowledge directly misguides subsequent reasoning and drives the final incorrect outcome. Unlike τ_8 , the error at τ_{12} cannot be corrected through downstream verification, highlighting its role as the decisive upstream failure.

The DCFA framework identifies this decisive error through a two-stage reasoning process. First, GCA constructs a step-level causal-inspired dependency graph centered on minor deviations, capturing high-level dependencies and propagating influence to generate an initial hypothesis, here pointing to τ_9 . Second, LCE performs a local neighborhood analysis around the hypothesis, employing counterfactual-inspired simulations to quantify the impact of each step on the final outcome. By eval-

uating the potential corrections in context, LCE isolates τ_{12} as the only step whose modification could prevent the error, thereby pinpointing the true decisive error.

This dual-view strategy, combining global reasoning over a causal-inspired dependency structure with local counterfactual-inspired validation, allows DCFA to transition from broad dependency mapping to precise error localization, effectively distinguishing contributory minor deviations from decisive errors. The approach not only enhances the interpretability of multi-agent reasoning traces but also identifies critical interactions whose correction is most likely to improve the final outcome, demonstrating clear advantages over purely global or local attribution methods.

I Bias Reduction

To further validate the robustness of our causal-inspired attribution, we examine DCFA’s capability to mitigate bias in decisive error localization compared with baselines. Here, *bias* refers to the systematic deviation of predicted steps from the ground-truth decisive error, which can obscure the actual failure source and mislead subsequent debugging (Li et al., 2022; Shrotriya et al., 2025).

We compare DCFA with the representative baseline, ECHO, on both Algorithm-generated and Hand-crafted traces from the Who&When benchmark (Zhang et al., 2025). For each trace, we quantify the deviation between the predicted and true causal-step indices:

$$d_{\text{DCFA}} = |t_{\text{DCFA}}^* - t^\dagger|, \quad d_{\text{Baseline}} = |t_{\text{Baseline}}^* - t^\dagger| \quad (17)$$

where t_{DCFA} and t_{Baseline} are predicted indices, and $t^\dagger$ denotes the ground-truth step. A smaller deviation implies lower bias and more reliable reasoning.

Figure 8 visualizes the deviation distributions for DCFA and ECHO. Across both DeepSeek-R1-671B and Gemini-2.5-pro backends, DCFA exhibits a notably sharper and more centralized distribution around zero. Overall, DCFA exhibits a smaller failure attribution offset than ECHO, indicating more accurate and less biased localization of the true decisive errors. This indicates that DCFA predictions consistently align closer to the ground-truth decisive error. Such improvement stems from the both global and local view causal-inspired enhancement of the DCFA, preventing error accumulation in step-level reasoning. Overall, the results

demonstrate that DCFA reduces attribution bias, leading to more stable and interpretable failure diagnostics across synthetic and real-world traces.

J Related Works

J.1 LLM-based Multi-Agent Systems and System Failure

The recent progress of Large Language Model (LLM)-based agents has accelerated the development of Multi-Agent Systems (MAS), which have emerged as a promising paradigm for addressing complex, multi-step tasks that exceed the capabilities and token limitations of single, monolithic LLMs (Yang et al., 2026; Sun et al., 2025; Pei et al., 2025; Xia et al., 2025). The principles underlying MAS, such as task decomposition, inter-agent communication, and emergent collective reasoning. Recent surveys have proposed systematic taxonomies of agent systems architectures (Wu et al., 2023; Fourney et al., 2024), identifying core functional components required for effective coordination and execution. These components generally include explicit role assignment, hierarchical or iterative planning mechanisms, structured communication protocols, externalized memory modules, and evaluation and feedback mechanisms (Yan et al., 2025).

Despite rapid progress, current agent systems remain limited in robustness, interpretability, and stability (Zhang et al., 2025; Cemri et al., 2026). Agents often exhibit brittle coordination, unstable reasoning trajectories, and cascading errors, particularly in long-horizon or highly interactive environments (Zhang et al., 2025; Cemri et al., 2026). Benchmarking efforts such as AgentBench (Liu et al., 2024b) empirically demonstrate these weaknesses, showing that even state-of-the-art systems struggle to maintain coherence and consistency in collaborative, multi-step tasks. The recurrence of these failures exposes a critical research gap: understanding why and how LLM-based agent systems deviate from their intended execution pathways.

J.2 Failure Attribution in LLM-based Multi-agent Systems

Prior work has studied error analysis in agentic systems. AgentErrorBench with AgentDebug (Zhu et al., 2025) locates mistakes that trigger cascading reasoning distortions in human-agent interactions in single-agent settings, whereas TRAIL (Deshpande et al., 2025) analyzes errors in execution trajectories in multi-agent settings. These definitions,

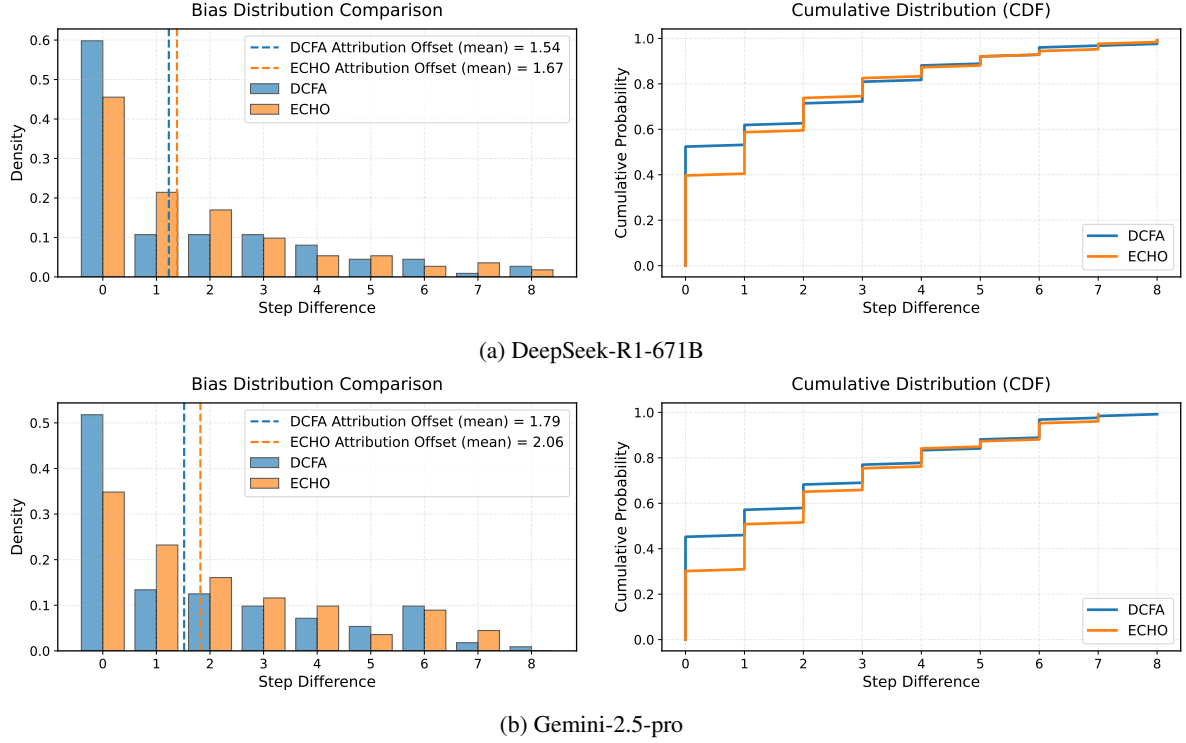


Figure 8: Distribution of step-level deviations between predicted and true decisive errors. DCFA shows more concentrated distributions around zero, indicating substantially reduced bias compared to ECHO.

however, do not directly target the most actionable points for repairing task failures. Meanwhile, unlike single-agent systems with sequential execution and full information access, multi-agent systems involve heterogeneous roles, complex interaction topologies, partial observability, and non-trivial communication, producing intricate and interdependent dependency chains (Cemri et al., 2026; Fourney et al., 2024; Wu et al., 2023). Thus, the focus shifts to identifying the earliest mistake whose correction could reverse overall system failure in multi-agent systems. The mistake, termed the decisive error (Cemri et al., 2026; Fourney et al., 2024; Wu et al., 2023), represents the key point for restoring task success.

Recent research has begun to explore failure attribution in multi-agent systems using LLMs as diagnostic detectors (Zhang et al., 2025, 2026). Existing methods can be categorized into fine-tuning-based (Zhang et al., 2026) and instruction-based (Zhang et al., 2025; Cemri et al., 2026) paradigms. The fine-tuning-based approach, exemplified by AgenTracer (Zhang et al., 2026), constructs specialized datasets to train dedicated models for failure diagnosis. While this approach offers strong controllability, it also incurs substantial costs for annotation, training, and maintenance.

Instruction-based methods (Zhang et al., 2025; Cemri et al., 2026), in contrast, rely on carefully designed prompts to guide LLMs in analyzing multi-agent system traces and identifying the decisive errors. For example, the Step-by-Step algorithm in the Who&When benchmark (Zhang et al., 2025) formulates failure attribution as a sequential inspection process that progressively checks each interaction. Another algorithm in the benchmark, Binary-Search, adopts a divide-and-conquer strategy, recursively narrowing the trace scope until the decisive error is pinpointed. Additionally, Banerjee et al. (Banerjee et al., 2025) propose hierarchical context representations and multi-perspective consensus to improve attribution reliability, while Weng et al. (West et al., 2025) introduce A2P, a causal-guided attribution framework powered by the DeepScientist AI system.

Despite these advances, current methods (West et al., 2025; Banerjee et al., 2025) rely primarily on the vanilla reasoning capabilities of LLMs, which often restricts their focus to superficial deviations and leads to performance degradation when processing long system traces. These limitations frequently result in inaccurate or incomplete attribution. To address these challenges, we propose a unified framework that incorporates structured

reasoning and extracts local dependency chains to enable robust and reliable failure attribution in LLM-based multi-agent systems.

J.3 Causal Reasoning and Counterfactual Analysis via LLM

Recent advances have increasingly explored the use of large language models (LLMs) for causal reasoning and counterfactual-inspired analysis, including discovering, representing, and simulating causal relationships (Su et al., 2025; Cheng et al., 2025; Luo et al., 2024). This line of research leverages the linguistic and reasoning capabilities of LLMs to extract and reason about causal structures from unstructured text. These studies show that LLMs can effectively extract events and propose candidate causal links with high recall, serving as powerful front-end modules for event causality analysis (Liu et al., 2024a; Wang et al., 2024).

To improve reliability, several works (Tong et al., 2024; Cohrs et al., 2025; Ban et al., 2025; Liu et al., 2025) integrate LLM-derived evidence with algorithmic estimators, aggregating multiple causal hypotheses or graph structures and taking their intersection to enhance robustness and filter out spurious relations. This hybrid approach demonstrates that LLMs can serve as high-level causal reasoning modules, while traditional estimators provide quantitative validation. Further research explores the use of LLMs as informative priors for causal graph discovery. Recent studies demonstrate that LLMs encode rich domain and commonsense knowledge that can guide structure learning and improve interpretability (Darvariu et al., 2024; Jiralerspong et al.). By injecting LLM-derived constraints or edge probabilities into graph search algorithms, these methods reduce sample complexity and produce more plausible causal graphs for downstream causal analysis.

Beyond causal structure discovery, causal inference and counterfactual simulation have been explored as mechanisms for testing causal hypotheses (Liu et al., 2025; Zhu et al., 2024; Verma et al., 2025). Causal inference provides formal tools for reasoning about the effects of interventions (Imai and Nakamura, 2026), while counterfactual simulation assesses how altering an event or reasoning step might change subsequent outcomes. Recent works apply such techniques to LLM reasoning chains, using generated interventions to evaluate faithfulness and sensitivity to hypothetical changes in chain-of-thought reasoning (Tutek et al., 2025;

Yu et al., 2025). These findings suggest that LLMs can both represent causal structures and support the simulation of hypothetical interventions for evaluating outcome changes.

Inspired by these developments, our work adapts these ideas to failure attribution in LLM-based multi-agent systems. Rather than claiming formal causal inference, DCFA employs causal-inspired structured reasoning over system traces. Specifically, we construct a causal-inspired dependency graph to organize inter-agent dependencies and use counterfactual-inspired evaluation with LLM-mediated approximate interventions to assess how correcting candidate interactions may alter the final outcome. This dual-view design provides an interpretable basis for identifying the interaction that most plausibly constitutes the decisive error in long and unstructured multi-agent traces.

K Pseudocode Description of DCFA

This section briefly explains the pseudocode of the two modules in the DCFA framework.

Global Causal-inspired Attribution (GCA).

Algorithm 1 summarizes the overall global attribution procedure. The module first identifies candidate minor deviations in the system trace using the deviation identification process $\text{MDI}(\cdot)$, which extracts interactions that exhibit abnormal behaviors together with brief textual explanations. Based on these candidates, the causal-inspired dependency graph construction procedure $\text{CGC}(\cdot)$ organizes the trace into a directed causal-inspired dependency graph that encodes plausible dependencies between interactions. Finally, the hypothesis generation process $\text{HG}(\cdot)$ analyzes the trace, the detected deviations, and the constructed causal-inspired dependency graph to produce an initial hypothesis

Algorithm 1 Global Causal-inspired Attribution (GCA)

Input: trace $\mathcal{T}$, ground truth $\hat{\mathcal{Y}}$

Output: hypothesis (τ^*, r^*) , causal-inspired dependency graph $\mathcal{G}$

Minor Deviation Identification

$(\mathcal{T}^e, \mathcal{R}^e) \leftarrow \text{MDI}(\mathcal{T})$

Dependency Graph Construction

$\mathcal{G} \leftarrow \text{CGC}(\mathcal{T}, \mathcal{T}^e, \mathcal{R}^e)$

Hypothesis Generation

$(\tau^*, r^*) \leftarrow \text{HG}(\hat{\mathcal{Y}}, \mathcal{T}, \mathcal{T}^e, \mathcal{R}^e, \mathcal{G})$

return $\tau^*, r^*, \mathcal{G}$

(τ^*, r^*) for the decisive error.

Local Counterfactual-inspired Enhancement (LCE). Algorithm 2 refines the global hypothesis by performing a bidirectional greedy search on the causal-inspired dependency graph. The procedure first constructs a corrected trace using $\text{Correct}(\cdot)$ to provide a reference trajectory aligned with the ground-truth outcome. Starting from the hypothesis interaction τ^* , the algorithm iteratively explores downstream and upstream neighbors in two search loops. At each step, candidate interactions are evaluated using counterfactual-inspired interventions, and the interaction with the largest positive marginal improvement $\Delta\mathcal{S}$ becomes the next pivot. The union of visited interactions forms a compact dependency neighborhood, which is then evaluated

Algorithm 2 Local Counterfactual-inspired Enhancement (LCE)

Input: trace $\mathcal{T}$, ground truth $\hat{Y}$, causal-inspired dependency graph $\mathcal{G}$, hypothesis τ^*
Output: decisive error τ^*
 $\hat{\mathcal{T}} \leftarrow \text{Correct}(\hat{Y}, \mathcal{T}, \mathcal{G})$
 $p \leftarrow \text{index}(\tau^*)$
 Forward Search
while True **do**
 $\mathcal{N} \leftarrow \{\tau_j \mid (p, j) \in \mathcal{E}\}$
 evaluate $\Delta\mathcal{S}(\tau_j)$ for $\tau_j \in \mathcal{N}$
 $p' \leftarrow \arg \max_{\tau_j \in \mathcal{N}} \Delta\mathcal{S}(\tau_j)$
if $\Delta\mathcal{S}(p') > 0$ **then**
 $p \leftarrow p'$
else
break
end if
end while
 $p \leftarrow \text{index}(\tau^*)$
 Backward Search
while True **do**
 $\mathcal{N} \leftarrow \{\tau_k \mid (k, p) \in \mathcal{E}\}$
 evaluate $\Delta\mathcal{S}(\tau_k)$ for $\tau_k \in \mathcal{N}$
 $p' \leftarrow \arg \max_{\tau_k \in \mathcal{N}} \Delta\mathcal{S}(\tau_k)$
if $\Delta\mathcal{S}(p') > 0$ **then**
 $p \leftarrow p'$
else
break
end if
end while
 $\mathcal{T}^c \leftarrow \text{visited interactions}$
 $\tau^* \leftarrow \text{DED}(\hat{Y}, \mathcal{T}^c, \mathcal{G})$
return τ^*

by the decision function $\text{DED}(\cdot)$ to determine the final decisive error.

L Prompts for DCFA

This section presents the prompts used to implement the DCFA components described in Section 4. The prompts correspond to key reasoning steps in the Global Causal-inspired Attribution (GCA) and Local Counterfactual-inspired Enhancement (LCE) modules. For clarity and reproducibility, we provide representative prompt templates. Actual inputs (e.g., system traces, candidate events, and ground-truth answers) are dynamically inserted during execution.

Prompts for the GCA Module. The GCA module performs global structured reasoning over the full system trace to generate an initial hypothesis for the decisive error.

First, the model identifies candidate *minor deviations* and constructs a causal-inspired dependency graph over the system trace. The prompt instructs the model to analyze the interaction events extracted from the system trace, detect potential mistake events, and identify dependency relationships among events based on the criteria of *temporality*, *necessity*, and *sufficiency* defined in Section 4.1. This step jointly performs minor deviation identification and causal-inspired dependency graph construction, producing both the set of candidate deviation events and the directed dependency relationships among them. The prompt used for this step is shown in Figure 9.

After the candidate deviations and causal-inspired dependency graph are obtained, the GCA module performs global reasoning to generate an initial hypothesis of the decisive error. Given the system trace, the set of candidate minor deviations, the explanations for each deviation, and the constructed causal-inspired dependency graph, the model evaluates how deviations propagate through the interaction chain and selects the event that most plausibly explains the final failure outcome. The corresponding prompt is illustrated in Figure 10.

Prompts for the LCE Module. The LCE refines the hypothesis generated by GCA through localized counterfactual-inspired reasoning, comprising prompt-based steps for error selection.

First, in the counterfactual-inspired evaluation process, the model generates a corrected system trace. Given the original trace, the identified mis-

take events, and the ground-truth outcome, the prompt instructs the model to correct the erroneous events and causally affected downstream interactions while preserving the original event structure and formatting. This produces a fully corrected trace representing a counterfactual-inspired scenario in which the mistakes are fixed. The prompt is shown in Figure 11.

Next, for each counterfactual-inspired trace constructed during evaluation, the model simulates the corresponding system outcome based on the partially corrected interaction sequence. Starting from the corrected prefix and the remaining original interactions, the model reconstructs the downstream reasoning process and predicts the resulting final outcome. This prompt enables the model to estimate how correcting specific interactions affects the final result. The corresponding prompt is presented in Figure 12.

Finally, after the bidirectional dependency neighborhood search and counterfactual-inspired evaluation produce a set of candidate decisive error events, the model performs a final semantic decision and selection step. Given the candidate events, the causal-inspired dependency graph structure, and the ground-truth outcome, the model determines which interaction most plausibly constitutes the decisive error responsible for the observed system failure. The prompt is shown in Figure 13.

System Instruction:

You are an expert in log analysis and causal reasoning.

Given a multi-agent interaction log, identify potential mistake events and construct a causal graph among all events.

Input:

All Events JSON: {ALL_EVENTS_JSON}

Instructions:

1. Parse all events from the interaction log.
2. Identify potential mistake events. A mistake event is an atomic action that may cause task failure, including:
 - incorrect or fabricated information,
 - unreasonable assumptions when encountering obstacles (e.g., API failure),
 - technical execution failures,
 - unproductive or irrelevant actions,
 - other actions likely to cause failure.
3. For each mistake event, record its event_id and a brief mistake_reason.
4. Construct a causal graph over all events. For each candidate causal edge ($e_i \rightarrow e_j$), ensure:
 - Temporality: e_i precedes e_j in the event sequence.
 - Necessity: removing e_i would cause e_j to fail to occur under the counterfactual intervention $do(\neg e_i)$.
 - Sufficiency: enforcing e_i would plausibly induce the occurrence of e_j under the intervention $do(e_i)$.
5. Determine causal relationships between earlier events and later events across all steps.

Output (JSON only):

```
{
  "mistake_events": {
    "event_<step_id>_1": "reason for mistake"
  },
  "causal_graph": {
    "step_id": {
      "event_dict": {
        "event_<step_id>_1": {
          "cause": ["event_X"],
          "effect": ["event_Y"]
        }
      }
    }
  }
}
```

Figure 9: Prompt used for minor deviation identification and causal-inspired dependency graph construction. The model identifies surface-level mistake events as minor deviations, and establishes dependency relations among them according to temporality, necessity, and sufficiency criteria.

System Instruction:

You are an AI assistant specialized in analyzing failures in multi-agent systems.

Your task is to identify the single decisive root-cause event responsible for the system's final failure by analyzing interaction logs, detected mistake events, and a causal graph.

Input

- Ground Truth: {GROUND_TRUTH}
- All Events JSON: {ALL_EVENTS_JSON}
- Mistake Events Reasons: {MISTAKE_EVENTS_REASONS_JSON}
- Causal Graph JSON: {CAUSAL_GRAPH_JSON}

Instructions

1. Identify the Mistake Agent

Using All Events JSON and Mistake Events Reasons, determine which agent initiated the incorrect causal chain or committed the most critical error.

2. Locate Obvious Mistakes

From Mistake Events Reasons, identify obvious mistakes such as tool failures, incorrect outputs, or explicit error messages.

3. Trace the Causal Path

Using the Causal Graph JSON, trace the causal chain from these mistakes to the system's final output.

Discard mistakes that are isolated or later corrected.

4. Determine the True Root Cause

Identify the decisive event that ultimately determined the incorrect final outcome.

This may be:

- Upstream non-obvious error (e.g., incorrect assumptions, fabricated information, wrong tool selection)
- Downstream decisive error (e.g., flawed computation or incorrect final decision)

5. Provide Reasoning

Explain the reasoning process, including:

- the initially identified obvious mistake,
- why it is not the true root cause,
- the inferred decisive root cause along the causal chain.

Output (JSON only):

```
{
  "log_answer": "<string: final incorrect answer in the trace>",
  "root_cause_event": "<string: event ID of the decisive root cause>",
  "event_step": "<string: step index where the root cause occurs>",
  "reason": "<string: concise explanation of the causal reasoning>"
}
```

Figure 10: Prompt used for global causal-inspired structured reasoning and hypothesis generation. Given the system trace, candidate minor deviations, their explanations, and the constructed causal-inspired dependency graph, the model performs global reasoning over the dependency structure to analyze how errors may propagate and proposes an initial hypothesis for the decisive error.

System Instruction:

You are an expert in causal reasoning and log correction.

Input:

GROUND_TRUTH: {GROUND_TRUTH}

Mistake Event Reasons: {MISTAKE_EVENT_REASONS}

All Extracted Events: {ALL_EVENTS}

Causal Graph: {CAUSAL_GRAPH_JSON}

Task:

1. Identify why the root cause event leads to task failure based on the mistake reasons.
2. Correct the mistake events and any causally affected events using the ground truth.
3. Keep the event sequence structure unchanged.
4. Only modify factual elements (names, numbers, labels). Do not introduce new events.

Output:

Return ONLY a JSON object containing the corrected version of 'All Extracted Events' with the same format as the input.

Output (JSON only):

```
{
  "step_id": {
    "agent_name": "...",
    "event_dict": {
      "event_id": "corrected event"
    }
  }
}
```

Figure 11: Prompt used for causal-inspired log correction. The model corrects mistake events and causally affected events using ground-truth information while preserving the original event structure and format.

System Instruction:
 You are given a partially corrected interaction log, a question, and the identified root-cause event that originally caused task failure. Only the root cause event and its directly affected events have been corrected; other events may still contain errors.

Input:
 Modified Log: {MODIFIED_LOG}
 Root Cause Event: {ROOT_CAUSE_EVENT}

Task:

1. Identify the corrected root cause event and its directly affected events.
2. Based on these corrections, reconstruct the downstream interaction sequence and fix any remaining dependent errors.
3. Infer the correct final answer from the corrected event sequence.
4. Briefly explain how the root cause led to the original incorrect answer and how the correction resolves it.

Output (JSON only):

```
{
  "reason": "...",
  "answer": "..."
}
```

Figure 12: Prompt used for counterfactual-inspired outcome simulation. Given a partially corrected interaction trace, the model reconstructs the downstream reasoning process and predicts the resulting final outcome.

System Instruction:
 You are an AI assistant specialized in multi-agent failure attribution.
 Your goal is to identify the single root-cause event that ultimately led to the system failure.

Input:
 GROUND_TRUTH: {GROUND_TRUTH}
 CAUSAL_EVENTS: {CAUSAL_EVENTS_JSON}
 MISTAKE_EVENTS: {MISTAKE_EVENTS_REASONS_JSON}

Task:

1. Examine all identified mistake events and understand how each contributes to the failure.
2. Analyze the causal chain among these events.
3. Identify the single event that is causally decisive for the final incorrect outcome.
 - This may be an upstream error (e.g., incorrect assumption, fabricated data).
 - Or a downstream decisive error (e.g., incorrect computation or final decision).
4. Provide a concise explanation of why this event is the true root cause.

Output (JSON only):

```
{
  "root_cause_event": "<event_id>",
  "reason": "<explanation of why this event is the root cause>"
}
```

Figure 13: Prompt used for decisive error identification. The model analyzes candidate mistake events and selects the single event that is causally decisive for the final system failure.